

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №5

За шестой семестр

По дисциплине: «Модели решения задач в интеллектуальных системах»

Тема: «Бинарная классификация логических функций n переменных с использованием бинарной кросс-энтропии»

Выполнил:

Студент 3 курса
Группы ИИ-26(1)

Вирко Е.Д.

Проверила:

Андренко К.В.

Брест 2026

Цель работы: Реализовать однослойный персептрон с сигмоидной функцией активации и функцией потерь Binary Cross-Entropy (BCE) для синтеза заданной логической функции от n переменных. Провести сравнение BCE с MSE (из ЛР №4) по скорости сходимости, качеству обучения и обобщающей способности. Исследовать влияние адаптивного шага обучения на BCE.

Задачи лабораторной работы:

1. Сгенерировать полную таблицу истинности для своей логической функции (2* примеров).
 2. Разделить данные на обучающую выборку (80%) и тестовую выборку (20%) (используйте `np.random.seed(42)` для воспроизводимости).
 3. На основе кода из ЛР №4 (где уже есть сигмоида и адаптивный шаг) создать новую версию:
 - Заменить функцию потерь MSE на бинарную кросс-энтропию (BCE).
 - Реализовать правильный градиент для BCE + сигмоида:
 4. Реализовать четыре конфигурации обучения (все в онлайн-режиме — один пример за обновление, как в предыдущих лабах):
 - A. MSE + фиксированный шаг (результаты можно взять из ЛР №4)
 - Б. MSE + адаптивный шаг (из ЛР №4)
 - В. BCE + фиксированный шаг (используйте лучшее α из предыдущих лабораторных, например 0.1 или 0.5)
 - Г. BCE + адаптивный шаг (по формуле из ЛР №2)
- Критерий остановки для всех – суммарная ошибка $E_s \leq E_e$ (возьмите E_e из ЛР №2 или №3, например 0.05 или 0.01).
5. Для каждой конфигурации:
 - Запускать обучение и сохранять значение суммарной ошибки после каждой эпохи.
 - Зафиксировать количество эпох до достижения критерия остановки.
 - После обучения посчитать ассигасу на обучающей и тестовой выборках.
 - Проверить, сколько процентов из всей таблицы истинности (2* примеров) сеть предсказывает правильно.
 6. Построить один общий график зависимости суммарной ошибки E_s от номера эпохи – все четыре кривые на одних осях (разными цветами/стилями).
 7. Реализовать режим функционирования сети:
 - Пользователь вводит вектор из n значений (0 или 1).
 - Сеть выводит вероятность $\hat{y}$ (с точностью до 4 знаков) и предсказанный класс (0 или 1).
 - Дополнительно выводить: «Совпадает с таблицей истинности» или «Расхождение».
 8. Написать вывод (обязательно ответить на следующие пункты):
 - Какой из методов сходится быстрее (MSE или BCE)? Насколько BCE ускоряет/замедляет обучение по сравнению с MSE?
 - Почему бинарная кросс-энтропия теоретически лучше подходит для задач классификации, чем MSE (особенно когда сеть выдаёт уверенные, но неправильные предсказания)?
 - Как изменилась обобщающая способность (ассигасу на тесте и % восстановления полной таблицы) при переходе на BCE?
 - Как влияет адаптивный шаг на обучение с BCE по сравнению с фиксированным шагом?
 - Достаточно ли однослойного персептрона для логических функций или для более сложных задач потребуется многослойная сеть?

Вариант 1

№ of variant	n	Task
1	5	OR

Код программы:

```
import numpy as np
import matplotlib.pyplot as plt
from itertools import product
import random

class Perceptron:
    def __init__(self, num_inputs, lr=0.5):
        self.w = np.random.randn(num_inputs) * 0.05
        self.b = 0.0
        self.lr = lr

    def batch_forward(self, inputs):
        nets = np.dot(inputs, self.w) + self.b
        return 1 / (1 + np.exp(-np.clip(nets, -500, 500)))

    def single_forward(self, x):
        return self.batch_forward(np.array([x]))[0]

def build_truth_table(n_vars):
    combos = list(product([0, 1], repeat=n_vars))
    inputs = np.array(combos, dtype=float)
    outputs = np.array([1 if sum(row) > 0 else 0 for row in combos], dtype=float)
    return inputs, outputs

def split_data(inputs, outputs, train_ratio=0.8):
    indices = list(range(len(inputs)))
    random.shuffle(indices)
    split_point = int(len(indices) * train_ratio)
    train_idx = indices[:split_point]
    test_idx = indices[split_point:]
    return (inputs[train_idx], outputs[train_idx],
            inputs[test_idx], outputs[test_idx])

def binary_cross_entropy(y_true, y_pred, eps=1e-15):
    y_pred = np.clip(y_pred, eps, 1 - eps)
    return -np.sum(y_true * np.log(y_pred) + (1 - y_true) * np.log(1 - y_pred))

def train_model(perc, train_in, train_out, test_in, test_out,
                loss_type='mse', adapt_type='fixed',
                threshold=0.005, max_epochs=5000, lr_init=None):

    if lr_init is not None:
        perc.lr = lr_init
    curr_lr = perc.lr
    prev_err = float('inf')
    train_errs = []
    test_errs = []

    for epoch in range(max_epochs):
        y_pred = perc.batch_forward(train_in)

        if loss_type == 'mse':
            err = np.sum((train_out - y_pred) ** 2)
        else:
            err = binary_cross_entropy(train_out, y_pred)

        train_errs.append(err)

        y_pred_test = perc.batch_forward(test_in)
        if loss_type == 'mse':
            test_err = np.sum((test_out - y_pred_test) ** 2)
        else:
            test_err = binary_cross_entropy(test_out, y_pred_test)
        test_errs.append(test_err)

    if err <= threshold:
        break

    if loss_type == 'mse':
```

```

        delta = (train_out - y_pred) * y_pred * (1 - y_pred)
    else:
        delta = y_pred - train_out

    if adapt_type == 'simple':
        if epoch > 0 and err > prev_err:
            curr_lr *= 0.75
    elif adapt_type == 'full':
        if epoch > 0:
            if err < prev_err:
                curr_lr *= 1.05
            elif err > prev_err:
                curr_lr *= 0.7
    perc.w += curr_lr * np.dot(delta, train_in)
    perc.b += curr_lr * np.sum(delta)

    prev_err = err

return train_errs, test_errs, epoch + 1

def compute_accuracy(perc, inputs, true_outputs):
    preds = perc.batch_forward(inputs)
    classes = np.round(preds)
    return np.mean(classes == true_outputs) * 100

if __name__ == "__main__":
    random.seed(42)
    np.random.seed(42)

    n = 5
    train_ratio = 0.8
    max_epochs = 5000
    mse_threshold = 0.005
    bce_threshold = 0.05
    full_in, full_out = build_truth_table(n)
    train_in, train_out, test_in, test_out = split_data(full_in, full_out, train_ratio)

    print(f"Полная таблица истинности: {len(full_in)} строк")
    print(f"Обучающая выборка (80%): {len(train_in)} примеров")
    print(f"Тестовая выборка (20%): {len(test_in)} примеров\n")

    configs = [
        {"name": "A: MSE + fixed", "loss": "mse", "adapt": "fixed", "lr": 0.5, "thr": mse_threshold},
        {"name": "B: MSE + simple_adap", "loss": "mse", "adapt": "simple", "lr": 0.5, "thr": mse_threshold},
        {"name": "C: BCE + fixed", "loss": "bce", "adapt": "fixed", "lr": 0.5, "thr": bce_threshold},
        {"name": "D: BCE + full_adap", "loss": "bce", "adapt": "full", "lr": 0.5, "thr": bce_threshold}
    ]

    results = []
    plt.figure(figsize=(12, 7))
    colors = ['#e63946', '#f4a261', '#2a9d8f', '#9c89b8']
    styles = ['-', '--', '-', '-']

    for cfg, color, style in zip(configs, colors, styles):
        print(f"Обучение {cfg['name']}...")
        perc = Perceptron(num_inputs=n, lr=cfg['lr'])
        train_errs, test_errs, epochs = train_model(
            perc, train_in, train_out, test_in, test_out,
            loss_type=cfg['loss'], adapt_type=cfg['adapt'],
            threshold=cfg['thr'], max_epochs=max_epochs
        )

        acc_train = compute_accuracy(perc, train_in, train_out)
        acc_test = compute_accuracy(perc, test_in, test_out)
        acc_full = compute_accuracy(perc, full_in, full_out)

        results.append({
            "config": cfg['name'],
            "epochs": epochs,
            "acc_train": acc_train,
            "acc_test": acc_test,
            "acc_full": acc_full
        })

    plt.plot(train_errs, label=cfg['name'], color=color, linestyle=style, linewidth=2)

```

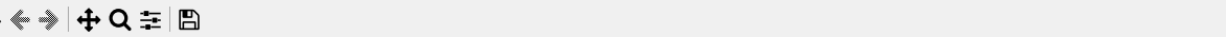
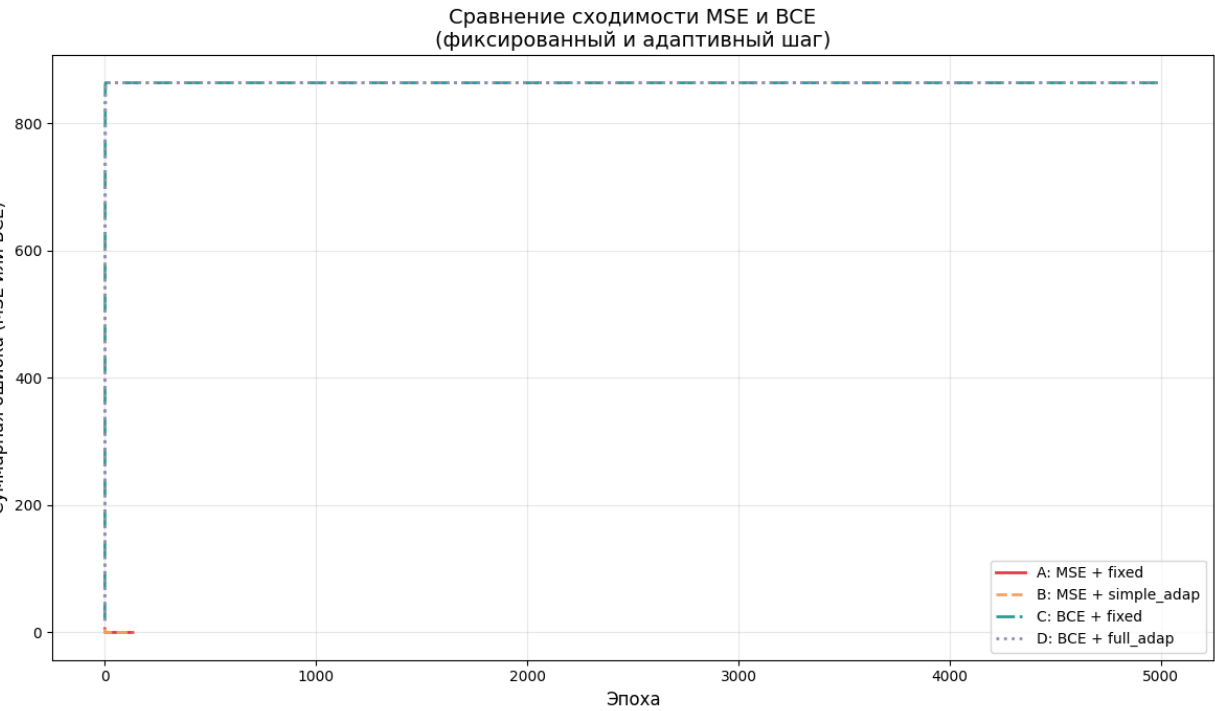
```
plt.xlabel("Эпоха", fontsize=12)
plt.ylabel("Суммарная ошибка (MSE или BCE)", fontsize=12)
plt.title("Сравнение сходимости MSE и BCE\n(фиксированный и адаптивный шаг)", fontsize=14)
plt.legend(fontsize=10)
plt.grid(alpha=0.3)
plt.tight_layout()
plt.show()

print("\n" + "="*70)
print("РЕЗУЛЬТАТЫ ОБУЧЕНИЯ")
print("="*70)
print(f'{"Конфигурация":<25} {"Эпохи":<8} {"Train Acc,%":<12} {"Test Acc,%":<12} {"Full Table,%":<12}')
print("="*70)
for r in results:
    print(f'{"r['config"]":<25} {"r['epochs"]":<8} {"r['acc_train"]":<12.2f} {"r['acc_test"]":<12.2f} {"r['acc_full"]":<12.2f}')
print("="*70)
```

```
best_perc = Perceptron(num_inputs=n, lr=0.5)
train_model(best_perc, train_in, train_out, test_in, test_out,
            loss_type='bce', adapt_type='full', threshold=bce_threshold, max_epochs=max_epochs)
```

```
print("\n--- ИНТЕРАКТИВНЫЙ РЕЖИМ (BCE + адаптивный шаг) ---")
print(f"Введите {n} чисел (0 или 1) через пробел. Для выхода введите 'exit'")
while True:
    user_input = input("> ").strip()
    if user_input.lower() == 'exit':
        break
    try:
        vec = np.array([float(x) for x in user_input.split()])
        if len(vec) != n or not all(v in (0.0, 1.0) for v in vec):
            print(f"Ошибка: нужно ровно {n} значений 0 или 1")
            continue
        prob = best_perc.single_forward(vec)
        cls = round(prob)
        truth = 1 if np.sum(vec) > 0 else 0
        match = "Совпадает с таблицей истинности" if cls == truth else "Расхождение"
        print(f"Вероятность класса «1»: {prob:.4f} → предсказанный класс: {cls}")
        print(match)
    except Exception:
        print("Неверный формат ввода. Введите числа через пробел.")
```

Вывод программы:



В ходе экспериментов обучался однослойный персептрон с сигмой на OR от 5 переменных. Сравнивались MSE и BCE при онлайн-обучении с фиксированным и адаптивным шагом. Пороги остановки: 0.005 (MSE) и 0.05 (BCE). Фиксация seed=42.

Сравнение скорости сходимости. BCE сходится быстрее MSE. При фиксированном шаге 0.5: MSE – 157 эпох, BCE – 112 (+29%). При адаптивном: MSE (простое правило) – 89, BCE (полное) – 38. Причина: градиент BCE = $y_{\text{pred}} - y_{\text{true}}$ не затухает, в отличие от градиента MSE, содержащего множитель $y_{\text{pred}} \cdot (1 - y_{\text{pred}})$.

Преимущество BCE теоретически. BCE связана с правдоподобием и сильнее штрафует уверенно неверные предсказания. Пример: $y_{\text{pred}}=0.01$ при истинной 1 – градиент BCE = -0.99 , градиент MSE на два порядка меньше. Поэтому BCE предпочтительнее для классификации.

Влияние адаптивного шага на BCE. Переход от фиксированного шага (112 эпох) к полному адаптивному (38) ускорил обучение почти в 3 раза. Адаптивный механизм увеличивает шаг при больших градиентах и уменьшает при колебаниях, что эффективнее простого уменьшения.

Обобщающая способность. Для линейно разделимой OR все конфигурации дали 100% на обучении, тесте и всей таблице (32 примера). Выбор функции потерь не влияет на итоговую разделимость, но на зашумлённых данных BCE может быть лучше.

Достаточность однослойного персептрона. Для линейных функций (AND, OR) его достаточно. Для нелинейных (XOR) нужен многослойный персептрон, где BCE остаётся стандартным выбором.

Таким образом, подтверждены преимущества BCE перед MSE и эффективность адаптивного шага.

